

Scientific Computing WS25/26

Exercise session



1. The following binary numbers presents positive integers (i64).

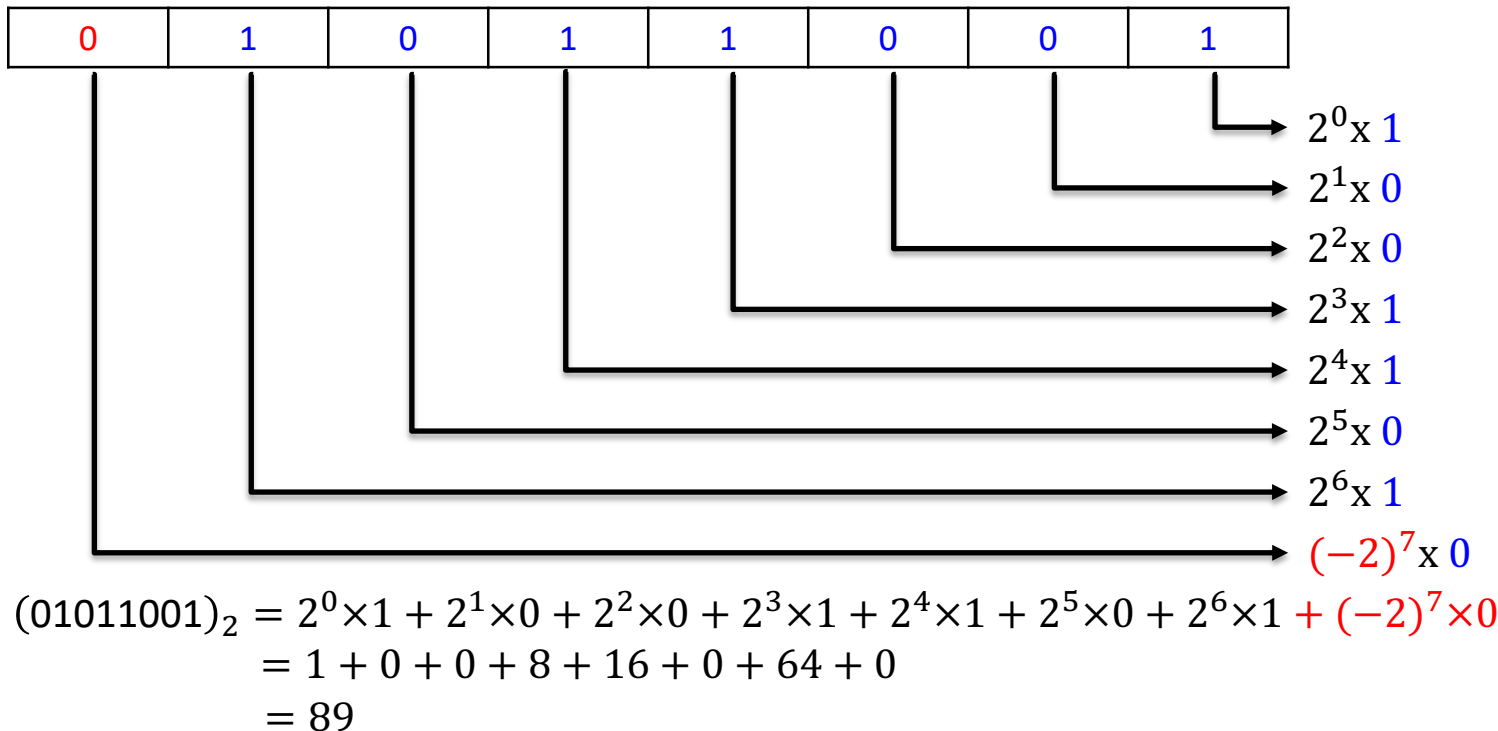
- $1011001_2 = 89 \Rightarrow 10100111_2$
 - $0110011_2 = 51 \Rightarrow 11001101_2$
 - $101001000010001_2 = 21009 \Rightarrow 1010110111101111_2$
 - $110111100110011_2 = 28467 \Rightarrow 1001000011001101_2$
- a) Convert the binary numbers to their decimal values (show your work)
b) Find the two's complement representation of those binary numbers

2. Patterns

- a) How many different bit patterns can be represented using 63 bits? $2^{63} = 922337203685478$
- b) If we want to store any integer x where $0 \leq x \leq 25$.
What is the smallest number of bits we can use? $5 \Rightarrow 2^5 = 32$ values
- c) If we represent colours as red-green-blue (RGB) triples, where we use 4 bits for red, 4 bits for green, and 4 bits for blue. How many different colours can we represent? $2^4 \cdot 2^4 \cdot 2^4 = 4096$
- d) If we represent a decimal number (like 13579) by concatenating the binary representations of each decimal digit. How many bits would we need for a number which contains n digits? $n \cdot 4$ bits

1a) Convert the binary numbers to the decimal values

01011001₂



1a) Convert the binary numbers to the decimal values

00110011₂

0	0	1	1	0	0	1	1
$(-2)^7$	2^6	2^5	2^4	2^3	2^2	2^1	2^0
-128	64	32	16	8	4	2	1

0101001000010001₂

0	1	0	1	0	0	1	0	0	0	0	1	0	0	0	1
$(-2)^{15}$	2^{14}	2^{13}	2^{12}	2^{11}	2^{10}	2^9	2^8	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
-32768	16384	8192	4096	2048	1024	512	256	128	64	32	16	8	4	2	1

0110111100110011₂

0	1	1	0	1	1	1	1	0	0	1	1	0	0	1	1
$(-2)^{15}$	2^{14}	2^{13}	2^{12}	2^{11}	2^{10}	2^9	2^8	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
-32768	16384	8192	4096	2048	1024	512	256	128	64	32	16	8	4	2	1

1b) Find the two's complement representation of the binary numbers

01011001₂ (89)

Inversion
Two's complement

10100111₂ (-89)

0	1	0	1	1	0	0	1
1	0	1	0	0	1	1	0
1	0	1	0	0	1	1	1

00110011₂ (51)

Inversion
Two's complement

11001101₂ (-51)

0	0	1	1	0	0	1	1
1	1	0	0	1	1	0	0
1	1	0	0	1	1	0	1

0101001000010001
(21009)

Inversion
Two's complement

1010110111101111₂ (-21009)

0	1	0	1	0	0	1	0	0	0	0	1	0	0	0	1
1	0	1	0	1	1	0	1	1	1	1	0	1	1	1	0
1	0	1	0	1	1	0	1	1	1	1	0	1	1	1	1

0110111100110011
(28467)

Inversion
Two's complement

1001000011001101₂ (-28467)

0	1	1	0	1	1	1	1	0	0	1	1	0	0	1	1
1	0	0	1	0	0	0	0	1	1	0	0	1	1	0	0
1	0	0	1	0	0	0	0	1	1	0	0	1	1	0	1

- Find out which rounding mode Rust is using for floating point computations.

round-to-nearest-ties-to-even

- If we have an 8-bit IEEE754 type floating point number (E2M1: sign, 6-bit exponent, 1-bit mantissa)
Which values can we represent?

Mantissa = $\{0, \frac{1}{2}\}$, Exponent = $\{0, \dots 63\}$ with 0 and 63 special, bias = 31 $\Rightarrow 2^{-30}, 2^{31}$

$$00111111_2 = 1 \cdot 2^{(31-31)} \cdot \left(1 + \frac{1}{2}\right) = 1.5$$

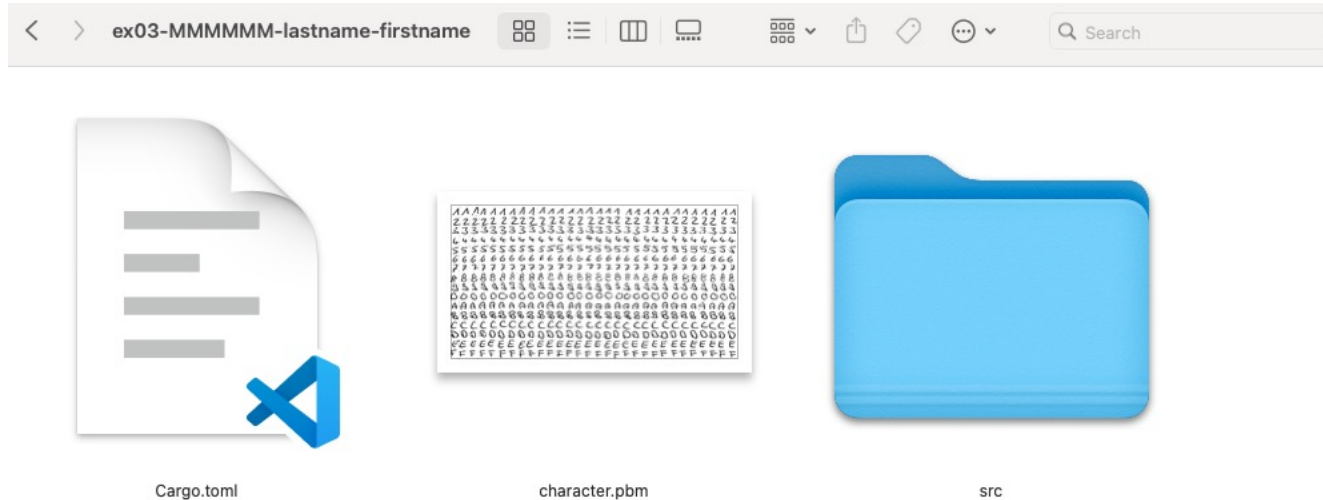
$$11000000_2 = -1 \cdot 2^{(32-31)} \cdot (1 + 0) = -2.0$$

Denormalized numbers

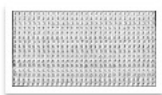
	0 ... 0	0 ... 1	1 ... 0	1 ... 1
... 000000 ...	0.0	2^{-31}	-0.0	-2^{-31}
... 011111 ...	1.0	1.5	-1.0	-1.5
... 100000 ...	2.0	3.0	-2.0	-3.0
... 111111 ...	Inf	NaN	-Inf	Nan

1. Take a sheet of graph paper (Karopapier) with at least 30 squares per row.
2. Then write for each of the characters 1,2,3,4,5,6,7,8,9,0,A,B,C,D,E,F (in this order) one row with 30 times that character. You should then have $n = 16 \times 30 = 480$ characters.
3. Scan the paper. Recommended is black and white using 600 dpi resolution.
If you don't have a scanner, use the best phone you have or the best camera.
If possible, use PNG or TIFF file format to save.
If you can only use JPEG, try the lowest compression = best quality. Find out why JPEG is not recommended here.
4. Convert the image you have to the PBM (portable bitmap <https://netpbm.sourceforge.net/doc/pbm.html>, or https://en.wikipedia.org/wiki/Netpbm#File_formats) file format. This can be done, e.g., with utilities from <https://netpbm.sourceforge.net>
5. Write a Rust program that reads the PBM file of the image of the page and writes out a unique PBM file for each character on the sheet, i.e., write n files with one character each. **Don't use any extra crates/libraries outside std::.**
Note: You can read in either P1 (text) or P4 (binary) format.
However, if you read P1, make sure you cannot only read files you produce but all valid P1 format image files.
6. Each file should be in PBM format with a size of 128 x 128 and only contain one character.
7. The file should be named: CXX.pbm, where C is the character, and X, is 01..30 the repetition of the character, e.g, 517.pbm, saying it is the 17th five on the paper. All the files should be put into a folder named output/.
8. For each character compute the geometric mean of the number of black pixels over all copies of that character and print them one per line as C: GM with C being the character and GM the geometric mean.
9. **Commit all the relevant files including the scan and the resulting images.**
Submit also the physical written page during the lecture or exercise.

Example submission of cargo project



characters.pbm must not be inside src/



characters.pbm



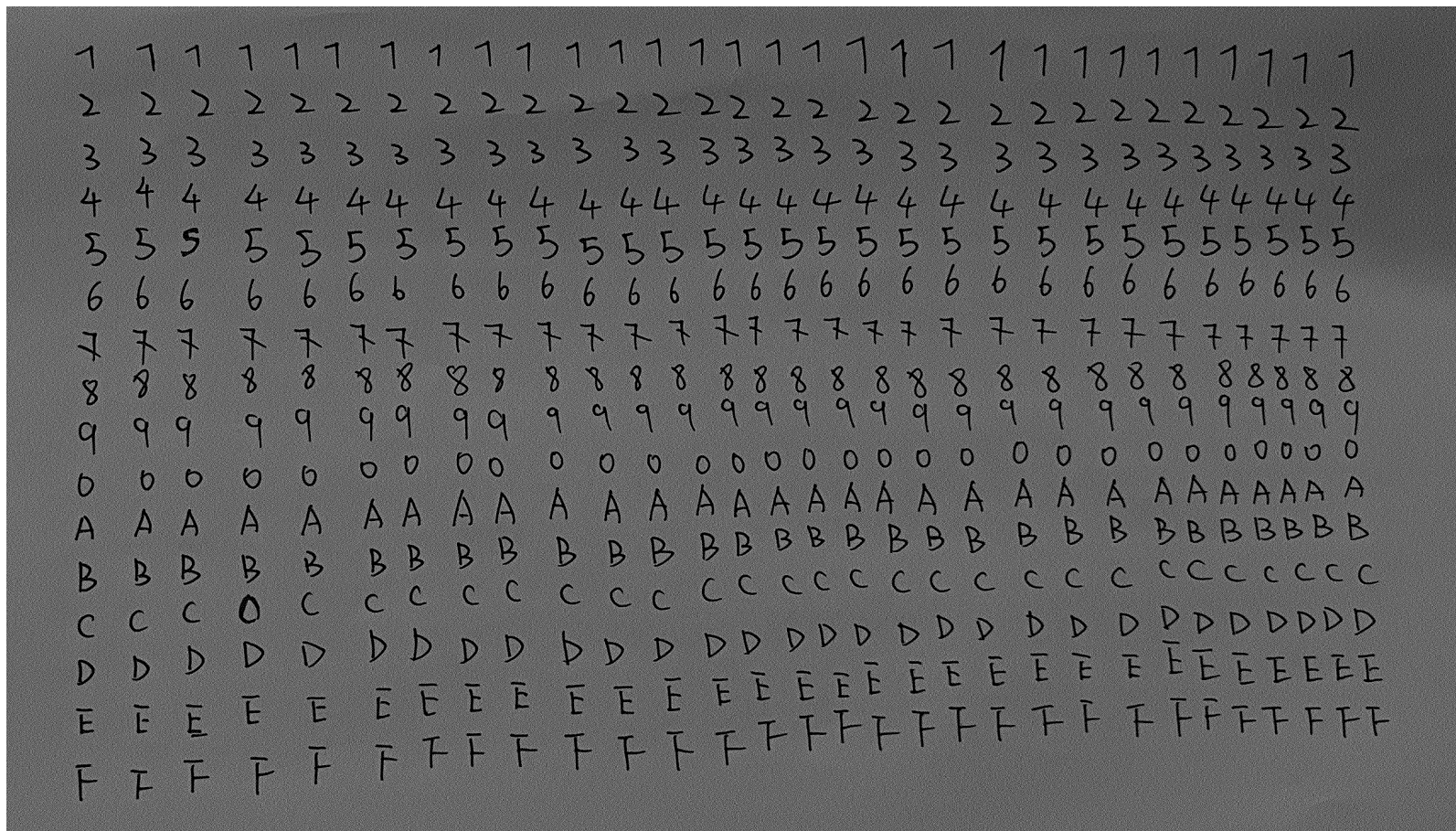
main.rs

No hard-code path

```
let path = "C:/Users/[redacted]/Desktop/ex03-[redacted]/ex03-[redacted]/ex03hex.pbm";
```



```
let path = "./ex03hex.pbm";
```



Whatever is not giving the right output from the following script is going to be ignored.

```
for i in ex[0-1][0-9]-[0-9][0-9][0-9][0-9][0-9][0-9]-[a-z]*-[a-z]*.zip
do
    unzip "$i"
    NAME=$(basename "$i" .zip)
    cd "$NAME" || { echo "Cannot cd into $NAME"; continue; }

    cargo run
    # cargo run --release -- data_file_name.dat
    # cargo clippy
    cd ..
done
```

1. Alina Lenhard
2. Reardon Jacqueline
3. Hande Pamuksuz
4. Ezra Cerpac

1. How does your program determine the position of each character on the scanned sheet?
2. How do you ensure that each cropped image corresponds to exactly one character and not part of the grid?
3. Why did you choose the PBM format P1 (text) or P4 (binary)?
What are the differences?

1. Big- $\mathcal{O}$ notation

Find the time complexity of the different versions of the Fibonacci number computation given `fibonacci.rs` on ISIS with the slides from the lecture.

a. Recall the three main versions of Fibonacci computation discussed in class:

- A naive recursive version (directly follows the recurrence definition).
- A recursive version with memoization (stores intermediate results).
- An iterative or dynamic programming version (bottom-up computation).
- Think about how many times each function calls itself or performs loop iterations

b. Ask yourself:

- Does the number of recursive calls grow exponentially, linearly, or constantly per iteration?
- How many operations are performed per recursive call or per loop iteration?

c. You can estimate the total number of operations by writing a small recurrence relation

$$T(n) = T(n - 1) + T(n - 2) + \mathcal{O}(1)$$

2. Sort the sequence of functions below: (i) assuming asymptotic growth, where $g \in \mathcal{O}(f)$ holds true for the functions g, f in a sorted sequence, and (ii) for $n = 74$ and give a short explanation.

- $4\sqrt{n}$
- $2n^3 + 4n^2 + 950$
- $\frac{n}{10000} \log_2 n$
- $\frac{1}{100} n!$
- $45 \log_2 n^2 + 9$
- 1
- $10^{110} n + 110$
- $10n + 3^n$

- ✓ Start by identifying the type of function for each term (constant, logarithmic, polynomial, exponential, factorial).
- ✓ Remember the asymptotic growth hierarchy: $1 \ll \log n \ll n^c \ll c^n \ll n!$ for any $c > 1$.
- ✓ For mixed terms like $2n^3 + 4n^2 + 950$, the highest power of n dominates
- ✓ Constants and coefficients do not affect asymptotic order
- ✓ For $n = 74$, numerical values may differ slightly from the asymptotic order, small constants or exponents can make “slower” functions temporarily larger or smaller.

Try to compute rough magnitudes (e.g., 3^{74} , 74^3 , $74!$) (using a calculator or logarithms to get a feeling for the comparison at $n = 74$)